

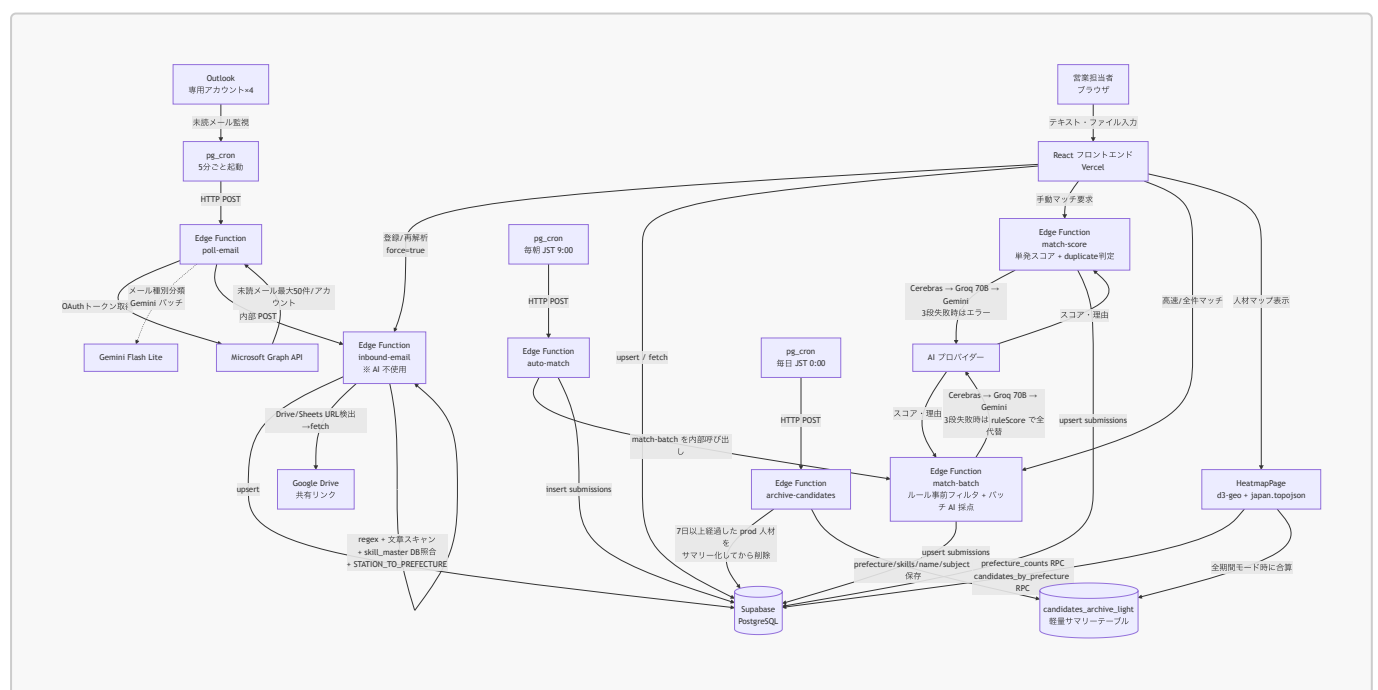
AkiNavi HR-AI

「案件の空き」と「人材リソース」をAIで自動マッチングするシステム。
ログイン不要・ニックネーム制・即日利用可能。

できること

機能	説明
AI マッチング	案件と人材の相性スコアと理由を AI が自動生成。手動 (match-score) / バッチ採点 (match-batch) / 自動 cron (auto-match) の 3 経路
人材登録	テキスト貼り付け・Excel・Word をアップロードするだけで自動解析・登録 (PDF / 画像は現状未対応)
案件登録	同上。メール本文や要件定義書をそのまま貼り付け OK。【場所】【単価】【時期】【備考】等のブラケット形式にも対応
メール自動取り込み	専用 Outlook アドレスを 5 分ごとにポーリング → 取得 → DB 保存 (AI 不使用・ルールベース)
人材マップ	登録済み人材を日本地図上の都道府県別ヒートマップで可視化。スキルフィルター・直近 7 日 / 全期間切替・都道府県クリックで受信メール一覧表示
デモ環境	本番データとは独立したデモ用データ環境 (?demo=KEY でトグル)。本番→デモコピー・スコア別 5 人生成

システム構成図



技術スタック

レイヤー	技術
フロントエンド	React 19, Vite 8, TypeScript, Tailwind CSS v4, TanStack Query v5
DB / バックエンド	Supabase (PostgreSQL, Edge Functions, pg_cron, pg_net)
AI (ブラウザ)	Gemini gemini-2.5-flash-lite (VITE_GEMINI_MODEL で変更可)。人材・案件登録 UI からは未使用 (Phase 4.11 で「AI で登録」廃止)
AI (サーバー・マッチング新方式)	match-batch : ルールベース事前フィルタ (スキル40/経験15/単価15/勤務地20/リモート10 = 100pt) → topN を 1 コール バッチ採点。Cerebras llama3.1-8b → Groq llama-3.3-70b-versatile → Gemini gemini-2.5-flash フォールバック。3 段失敗時はルールスコアで全代替
AI (サーバー・マッチング単発)	match-score : 上記と同じフォールバック順。 duplicateSuspected フラグ込みの単発スコア。理由は 150 字以内
AI (サーバー・自動マッチング)	auto-match (毎朝 JST 9:00 cron) : match-batch を内部呼び出し → 同じフォールバック順を継承
AI (サーバー・メール種別分類)	poll-email の同一受信箱判別: Gemini gemini-2.5-flash-lite バッチ (任意・既定は無効)
メール解析	AI 不使用 。regex (extractCandidateFieldsRegex + flexLabel) + 文章スキャン (extractFromProse) + skill_master DB 照合 (約 1,660 件) + 駅→都道府県マッピング (約 254 駅)
ファイル解析	xlsx (Excel) ・ mammoth (Word)。PDF と画像はテキスト解析対象外
人材マップ	d3-geo (Mercator 投影) + topojson-client + public/japan.topojson (416KB ・ 47 都道府県)。 prefecture_counts / candidates_by_prefecture RPC で SQL 集計
メール自動受信	Microsoft Graph API + Supabase pg_cron (完全無料 ・ Make.com 不要)
データアーカイブ	archive-candidates Edge Function (毎日 JST 0:00 cron)。7 日経過した prod 人材を candidates_archive_light にサマリー化してから DB 削除 (人材マップ全期間集計用)
Edge Function デプロイ事前検査	scripts/check-and-deploy-edge.sh (deno check で TS2304 を検知 → デプロイ中止)
デプロイ	Vercel (フロント) / Supabase (バックエンド)

レイヤー	技術
テスト	Vitest, React Testing Library, MSW + <code>scripts/verify_email_extraction.mjs</code> (メール抽出リグレッション)

無料枠の限界（現状）

メール解析が AI 非依存になったため、**メール取り込みは無料枠の影響を受けません**。AI を消費するのは「マッチング処理」と「メール種別分類（任意）」のみ。さらに `match-batch` でルールベース事前フィルタを噛ませているため、1 案件 = 1 AI コールに圧縮されます。

AI	役割	無料上限	備考
Cerebras <code>llama3.1-8b</code>	<code>match-batch</code> / <code>match-score</code> の 1 段目	実質無制限	軽量バッチ採点向け
Groq <code>llama-3.3-70b-versatile</code>	<code>match-batch</code> / <code>match-score</code> の 2 段目（精度重視）	500K tokens/日（JST 9:00 リセット）	マッチング数百～千案件/日が目安
Gemini <code>gemini-2.5-flash</code>	<code>match-batch</code> / <code>match-score</code> の最終フォールバック	プリペイド制（要チャージ）	1 バッチ ~3～5K tokens 程度
Gemini <code>gemini-2.5-flash-lite</code>	<code>poll-email</code> メール種別分類（任意）・ブラウザ補助	プリペイド制	既定 OFF

マッチング処理が天井。3 段すべて失敗してもルールスコアで全代替されるので、システム自体は止まらない。メール処理は規模に関係なく無料で永続稼働。

ローカル開発環境のセットアップ

前提条件

- ☐ Node.js 20 以上（`node -v` で確認）
- ☐ npm 9 以上（`npm -v` で確認）
- ☐ Git

手順

1. リポジトリをクローン

```
git clone https://github.com/kzmiyamura/akinavi-hr-ai.git
cd akinavi-hr-ai
```

2. 依存パッケージをインストール

```
npm install
```

3. 環境変数を設定

```
cp .env.example .env.local
```

`.env.local` を以下の内容で編集:

```
# Supabase (Dashboard → Settings → API から取得)
VITE_SUPABASE_URL=https://argizomylbolpqxgmvim.supabase.co
VITE_SUPABASE_ANON_KEY= (anon キーを貼り付け)

# Gemini (Google AI Studio から取得)
VITE_GEMINI_API_KEY= (APIキーを貼り付け)
VITE_AI_PROVIDER=gemini

# デモ環境の解除キー (任意・未設定でもOK)
VITE_DEMO_KEY= (任意の文字列)
```

4. Supabase の DB を初期化

Supabase Dashboard → SQL Editor で以下を**順番**に実行:

1. `supabase/schema.sql`
2. `supabase/migrations/` 配下の SQL を **ファイル名の昇順で全て実行**
 - 基本系: `add_skill_master.sql / seed_skill_master.sql / add_relevance_keywords.sql / add_box_columns.sql / add_resume_url.sql / add_attachments_bucket.sql`
 - RPC 系: `find_duplicate_candidates_rpc.sql / add_search_rpc.sql`
 - cron 系: `add_email_polling_cron.sql / add_auto_match_cron.sql / add_skill_cleanup_cron.sql / add_enrich_cron.sql` (`YOUR_PROJECT_REF` と `YOUR_SERVICE_ROLE_KEY` を実値に書き換え)
 - **Phase 4.10 / 4.11 / 4.12 で追加された新規マイグレーション:**
 1. `20260520130000_add_work_phases.sql` (IBM 系・ストレージ系・工程系 18 件)
 2. `20260521000000_add_search_scope.sql` (`search_candidates(p_scope)` 3 モード対応)
 3. `20260521210000_add_bigquery_and_cloud_dwh.sql` (DWH 系 10 件)
 4. `20260522_add_error_logs.sql` (フロント側エラーログ)
 5. `20260522_add_fetch_candidates_for_matching.sql` (MatchingPage RPC)
 6. `20260522_add_fetch_candidates_for_project.sql` (案件→人材 SQL 絞り込み RPC)
 7. `20260523_add_process_skills.sql` (テスト / 保守開発 / 保守運用 / 調査分析 4 件追加)
 8. `20260523_archive_light_table.sql` (人材マップ用 `candidates_archive_light` テーブル + 期間対応 `prefecture_counts` RPC)
 9. `20260523_prefecture_counts_rpc.sql` (人材マップ用 初版 RPC・後段で上書き)
 10. `20260523_normalize_prefecture.sql` (人材マップ用 `normalize_prefecture` 関数 + 最終版 `prefecture_counts` + `candidates_by_prefecture`)

11. `add_archive_candidates_cron.sql` (人材マップ用 7 日アーカイブ `pg_cron`。
`YOUR_PROJECT_REF / YOUR_SERVICE_ROLE_KEY` 置換が必要。旧 `delete-old-candidates` を `unschedule`)
12. 要追加 SQL (migration 漏れ対応) : `ALTER TABLE candidates_archive_light ADD COLUMN IF NOT EXISTS name text, ADD COLUMN IF NOT EXISTS subject text;` (`archive-candidates` Edge Function と `candidates_by_prefecture` RPC が両カラムを参照するため)

`schema.sql` の `candidate_skills.check_category` は 14 カテゴリへ更新済み。すべての `migrations/` を昇順で流すこと (新規環境構築時の必須手順)。人材マップ機能の詳細仕様は `docs/Heatmap.md` を参照。

5. 開発サーバーを起動

```
npm run dev
```

ブラウザで `http://localhost:5173` を開く。

本番デプロイ手順

Vercel (フロントエンド)

Vercel Dashboard → Environment Variables に以下を設定してから `main` ブランチへ push:

変数名	値
<code>VITE_SUPABASE_URL</code>	Supabase の URL
<code>VITE_SUPABASE_ANON_KEY</code>	Supabase の anon キー
<code>VITE_GEMINI_API_KEY</code>	Gemini API キー
<code>VITE_AI_PROVIDER</code>	<code>gemini</code>
<code>VITE_DEMO_KEY</code>	デモ解除キー (任意)

Supabase Edge Functions

```
supabase functions deploy inbound-email
supabase functions deploy poll-email
supabase functions deploy auto-match
supabase functions deploy match-score
supabase functions deploy match-batch # Phase 4.10 新規 (バッチ AI 採点)
supabase functions deploy archive-candidates # Phase 4.12 新規 (7日アーカイブ・人材マップ用)
supabase functions deploy microsoft-oauth
```

```
supabase functions deploy enrich-candidate
supabase functions deploy skill-master-cleanup
```

デプロイ前に型検査したい場合は `npm run check:edge <function>` (`scripts/check-and-deploy-edge.sh`) を使うと `deno check` で TS2304 (未定義変数) が出ていればデプロイを中止できる。`npm run deploy:edge <function>` で「型検査 + デプロイ」をまとめて実行。

Edge Functions Secrets (Supabase Dashboard → Edge Functions → Secrets)

Secret 名	用途	必須
GROQ_API_KEY	match-batch / match-score の 2 段目・poll-email 種別分類	◎
CEREBRAS_API_KEY	match-batch / match-score の 1 段目 (軽量・無料)	推奨
GEMINI_API_KEY	match-batch / match-score の最終フォールバック・poll-email 補助	◎
GRAPH_CLIENT_ID	Azure AD アプリのクライアント ID	◎
GRAPH_CLIENT_SECRET	Azure AD アプリのクライアントシークレット	◎
GRAPH_REFRESH_TOKEN_HUMAN	人材用メール (prod) のリフレッシュトークン	◎
GRAPH_REFRESH_TOKEN_PROJECT	案件用メール (prod) のリフレッシュトークン	◎
GRAPH_REFRESH_TOKEN_HUMAN_DEV	人材用メール (demo) のリフレッシュトークン	任意
GRAPH_REFRESH_TOKEN_PROJECT_DEV	案件用メール (demo) のリフレッシュトークン	任意
INBOUND_CALL_KEY	poll-email → inbound-email 呼び出し用 JWT (service_role キー)	◎
GOOGLE_SERVICE_ACCOUNT_JSON	Google Sheets/Drive (Box 連携キュー) アクセス用	Box 連携時
BOX_SPREADSHEET_ID	Box 連携キュー用スプレッドシート ID	Box 連携時

`inbound-email` 自体は AI を使わないので、メール取り込みだけ動かしたいなら `GROQ_API_KEY` 等は不要。マッチング系を使うときに必須になる。

pg_cron スケジュール登録

以下の SQL 内 `YOUR_PROJECT_REF` と `YOUR_SERVICE_ROLE_KEY` を実際の値に書き換えて SQL Editor で実行：

- `supabase/migrations/add_email_polling_cron.sql` (5 分ごと: poll-email)
- `supabase/migrations/add_auto_match_cron.sql` (毎朝 JST 9:00: auto-match)
- `supabase/migrations/add_skill_cleanup_cron.sql` (毎日 JST 3:00: skill-master-cleanup)

- `supabase/migrations/add_archive_candidates_cron.sql` (毎日 JST 0:00: archive-candidates。旧 `delete-old-candidates` を自動 `unschedule`)
- `supabase/migrations/add_enrich_cron.sql` (Box 連携時のみ)

アプリ設定 (`app_config` テーブル)

Edge Function 群の挙動はソースを書き換えずに `app_config` キーで切替できる。設定タブの UI から変更するか、Supabase SQL Editor で直接更新する。

キー	既定	内容
<code>inbound_project_enabled</code>	<code>false</code>	案件メールの解析と DB 保存を有効化。 <code>'true'</code> を設定すると <code>inbound-email</code> が <code>type=project</code> を処理 (既定は人材メールのみ取り込み)
<code>auto_match_enabled</code>	<code>true</code>	<code>auto-match</code> cron を有効化。 <code>'false'</code> で毎朝のバッチ実行をスキップ
<code>email_poll_mode</code>	<code>incremental</code>	<code>incremental</code> (未読のみ) か <code>full</code> (指定日以降全件)
<code>email_full_import_since</code>	(未設定)	<code>email_poll_mode=full</code> 時に取得を開始する ISO 日時
<code>email_classify_enabled</code>	<code>false</code>	同一受信箱に人材/案件が混在するとき、Gemini で <code>candidate/project/other</code> をバッチ分類
<code>matching_fast_max_candidates</code>	20	高速モード時の案件あたり候補者上限
<code>matching_fast_max_projects</code>	10	高速モード時の人材あたり案件上限
<code>candidate_retention_days</code>	7	人材データ保持日数 (旧データ自動削除用・運用判断で活用)
<code>app_memo</code>	(未設定)	営業引き継ぎ用フリーテキストメモ
<code>graph_rt_human_prod</code> ほか	—	Microsoft OAuth 連携で保存されるリフレッシュトークン (4 アカウント分)

`inbound-email` の即時マッチング切替は環境変数 (Supabase Secrets) で行う:

Secret	既定	内容
<code>AUTO_MATCH_ENABLED</code>	<code>false</code>	<code>true</code> で <code>inbound-email</code> 経由の即時自動マッチングも有効化 (普段は <code>cron</code> 経由のみ)

テスト実行

```
npm run test:run    # 全テスト (CI向け)
npm run test        # ウォッチモード (開発向け)
```

`scripts/verify_email_extraction.mjs` はメール解析の品質を一発でチェックする Node スクリプト (要 Node 20+)。Phoenix Technologies などの実メールフォーマットを使ったリグレーションケースを内蔵。

```
node scripts/verify_email_extraction.mjs
```

Edge Function デプロイ前検査

```
npm run check:edge inbound-email    # deno check のみ (TS2304 検知)
npm run deploy:edge inbound-email    # 型検査 + supabase functions deploy
```

`scripts/check-and-deploy-edge.sh` が `deno check` で TS2304 (未定義変数) を検知したら `deploy` を中止する。引数省略時は `inbound-email` を対象とする。

月次品質チェック

`.claude/skills/quality-check/SKILL.md` / `//quality-check` コマンドの手順に従う:

- Supabase Dashboard → Functions → inbound-email → Logs で `[station_unmapped]` を検索 → 集計
- `STATION_TO_PREFECTURE` に追記 → `npm run deploy:edge`
- `python3 scripts/skill_master_review.py` → 怪しい `source='ai'` スキルの削除候補 SQL を出力
- `[SKIP_IRRELEVANT]` ログを確認 (TRAINING_REPORT / PROJECT_SOLICITATION 等の誤投函パターン)
- AI コスト監視** (`ai_logs` テーブル) : モデル別・日次呼び出し数を集計し、Gemini 無料枠超過 / プリペイドクレジット枯渇 / フォールバック多発を検知

リファレンス

メール受信アドレス

5 分ごとに未読メールを自動取得・解析・DB 保存。処理完了後に既読マーク。

種別	アドレス	data_env
人材登録用 (本番)	<code>akinavi.hr.ai.voice.human@outlook.jp</code>	<code>prod</code>
案件登録用 (本番)	<code>akinavi.hr.ai.voice.project@outlook.jp</code>	<code>prod</code>
人材登録用 (デモ)	dev 用アカウント	<code>demo</code>

種別	アドレス	data_env
案件登録用（デモ）	dev 用アカウント	demo

メール自動受信フロー（Graph API ポーリング・AI 不使用）

```
pg_cron (5分ごと)
  ↓ HTTP POST
poll-email Edge Function
  | Graph API: リフレッシュトークン → アクセストークン取得（ローテーション保存）
  | GET /me/messages?$filter=isRead eq false (最大50件/アカウント、ページネーシ
  ョン継続)
  | メール種別分類（任意・既定 OFF）: Gemini バッチで candidate/project/other 判
  定
  |   → other はそのまま既読マークしてスキップ
  | 各メール処理:
  |   1. markAsRead (二重処理防止)
  |   2. 添付ファイル取得
  |   3. inbound-email へ POST
  |   4. 失敗時は markAsUnread (次回再試行)
  | 4アカウントを Promise.allSettled で並列処理
  ↓ 内部POST
inbound-email Edge Function (※ AI は呼ばない)
  | TRAINING_REPORT / PROJECT_SOLICITATION フィルタ (人材メールボックスへの誤投
  函を 200 OK でスキップ)
  | HTML → プレーンテキスト化 + HTML エンティティデコード
  | URL 除去・送信者署名除去 (誤マッチ対策)
  | Google Drive / Sheets / Docs URL 検出・自動取得
  | Word / Excel 添付 → テキスト変換 (PDF は Storage に保存するだけ)
  | 複数人材検出: 区切り線 (*****/— 等) で 1 メール = 複数候補者対応
  | skill_master DB 照合 (本文と添付で別ロジック、添付は上位 20 件・D/E評価除外)
  | extractCandidateFieldsRegex: 氏名・最寄駅・都道府県・経験年数・希望単価・参画
  時期・希望案件
  | extractFromProse: 役割・業界・リモート可否 (フェーズ表ヘッダーは除外)
  | 駅 → 都道府県マッピングで送信者署名由来の誤判定を上書き
  | 重複疑い: 名前一致 + スキル Jaccard ≥ 0.4 → duplicate_flag=true
  | DB保存 (candidates / projects / candidate_skills / ai_logs ※
  ai_logs.model='no-ai')
```

マッチング（AI 使用）

方式	トリガー	対象人材数上限	AI フォールバック順
match- batch	UI ボタン（高速/全件）または auto-match から内部呼び出し	高速モード: matching_fast_max_candidates （既定 20） auto-match: 案件 1 件あたり最大 40 名 (BATCH_AI_SIZE=20 × 2 リクエ スト)	Cerebras → Groq 70B → Gemini → 3 段失敗時は ルールスコ アで全代替

方式	トリガー	対象人材数上限	AI フォールバック順
match-score	手動（個別スコア確認・duplicate 検出）	1 ペア	Cerebras → Groq 70B → Gemini
auto-match	毎朝 JST 9:00 cron。 auto_match_enabled='false' でスキップ	直近 25 時間以内に登録された prod 案件 ×最大 40 名	match-batch 経由（Cerebras → Groq 70B → Gemini）

ルールベーススコア（match-batch の calcRuleScore ・ 0～100pt）

観点	配点	備考
スキル一致	最大 40pt	required 空のときは固定 +20pt
経験年数	最大 15pt	10年=15 / 7年=12 / 5年=8 / 3年=4 / 1年=2
単価	最大 15pt	予算未設定なら +15、範囲内 +15、上限+10% +8、上限+20% +3
勤務地	最大 20pt	同じ都道府県 / フルリモートで +20、居住地不明 +5
リモート	最大 10pt	リモート可・希望時 +10

AI 採点は **topN（既定 10）件のみ** バッチプロンプト 1 コールで実施し、残りはルールスコアのみで返す。マッチング理由は **150 字以内** で「必須スキル合致 → 経験年数 → 単価 → 勤務地リモート → 懸念点」の優先順。

ブラウザからのファイル解析フロー

ファイル選択（Excel / Word）

└ Excel → xlsx（SheetJS）で全シートを CSV 変換 → テキストエリアへ転記

└ Word → mammoth で本文抽出 → テキストエリアへ転記

↓

「登録」ボタン

↓

inbound-email Edge Function（force=true で DEDUP/SENDER_DAILY_LIMIT バイパス）

↓

regex + skill_master DB 照合 → candidates / projects に upsert

PDF と画像は現状未対応。 PDF を渡された場合は UI 側でエラー表示し処理を中断する。回避策: テキストを手動で貼り付ける、または PDF をページ画像化したうえで OCR テキストを手で貼り付ける。

データ環境（prod / demo）

同一 Supabase 内でデータを論理分離。data_env カラムでフィルタリング。

環境	用途	切替方法
prod	本番データ（実際の人材・案件）	デフォルト
demo	営業デモ用サンプルデータ	URLに <code>?demo=<VITE_DEMO_KEY></code> を付加

DB テーブル一覧

テーブル	用途
candidates	人材マスタ（ <code>data_env</code> で prod/demo 分離）。後述の主要カラム参照
projects	案件マスタ（ <code>data_env</code> で prod/demo 分離）
submissions	マッチング提案履歴（スコア・AI要約・ <code>ai_raw</code> に source タグ）
candidate_skills	スキルのカテゴリ別管理（14カテゴリ・CHECK制約）
candidates_archive_light	7 日以上経過した人材のサマリー（ <code>prefecture / skills / name / subject</code> ）。人材マップの「全期間」モード集計用。Phase 4.12 新規
ai_logs	AI 解析実行ログ（モデル名・所要時間・結果・エラー。 <code>model='no-ai'</code> でメール解析記録）
error_logs	フロントエンド側クライアントエラー (<code>page/message/stack/context/data_env/nickname</code>)
skill_master	スキル辞書（約 1,660 件 + AI 自動登録分。DWH/IBM/工程系を Phase 4.10 で強化）。 <code>aliases</code> で表記ゆれ吸収、 <code>match_count</code> で実績管理
relevance_keywords	関連度判定用キーワード（ <code>exclude / candidate / project</code> の 3 種別。現状は未使用・Phase 4.9 で <code>classifyInboundRelevance</code> 削除済み）
app_config	アプリ設定 / Graph API リフレッシュトークンのローテーション保存

candidates テーブルの主要カラム

カラム	内容
data_env	prod または demo
name	候補者名（イニシャル可）。抽出失敗時は '不明'
email / phone	候補者本人の連絡先（あれば）
skills (jsonb)	フラットなスキル名配列（ <code>skill_master</code> 由来）
experience_years	経験年数（年単位）
raw_profile (jsonb)	元メール本文・添付テキスト・抽出メタデータ・スキル分類などを保持
desired_rate	希望単価（例: "65万円以上"）
from_company	営業会社名（送信者署名から抽出）

カラム	内容
box_url / box_status	Box 共有 URL と取り込みステータス (pending / enriched / failed)
resume_url	Supabase Storage 上の履歴書 URL
drive_url	Google Drive 共有 URL
duplicate_flag	重複候補と判定された場合 true
merged_into	重複マージ先の候補者 ID

candidate_skills の14カテゴリ

カテゴリキー	内容・例
languages	Python, TypeScript, SQL 等
frameworks	React, Laravel, Django 等
libraries	jQuery, NumPy 等
os	Linux, Windows, macOS 等
databases	PostgreSQL, Redis, MongoDB 等
dwh	Snowflake, BigQuery, Tableau 等
clouds	AWS, GCP, Azure 等
infrastructures	Docker, Kubernetes, Terraform 等
tools	Git, Jira, Slack, JP1, Teraterm 等
methodologies	アジャイル, スクラム, PM 等
certifications	AWS認定, 情報処理技術者, ITパスポート 等
design	Figma, Illustrator, Photoshop 等
marketing	SEO, SNS運用, Web広告 等
others	上記以外

ディレクトリ構成

```
akinavi-hr-ai/
├── src/
│   ├── lib/
│   │   ├── ai/
│   │   │   ├── types.ts      # 型定義 (リクエスト・レスポンス)
│   │   │   ├── geminiProvider.ts # Gemini 実装 (マルチモーダル対応)
│   │   │   ├── openaiProvider.ts # OpenAI スタブ (未実装)
│   │   │   └── index.ts      # プロバイダー切替ファクトリ
│   │   └── db/              # DB 操作
│   └── candidates.ts
```

```

├── projects.ts
├── submissions.ts
├── emailSettings.ts
├── matchingSettings.ts
├── fileParser.ts      # Excel / Word テキスト抽出 (PDF / 画像 非対応)
├── dataEnv.ts         # prod/demo 環境切替
├── supabase.ts
├── pages/             # 各画面 (Matching / Candidate / Project /
Settings ほか)
├──                               # ※ History / Duplicate / Monitor は実装済みだ
がナビから非表示
├── components/        # 共通 UI (DemoSeedPanel /
DemoProjectCandidateGen 等)
├── supabase/
│   ├── schema.sql     # DB テーブル定義・RLS ポリシー
│   ├── migrations/    # 追加マイグレーション SQL (昇順で全て実行)
│   └── functions/
│       ├── inbound-email/      # メール解析 Edge Function (AI 不使用・regex
+ DB 照合)
│       ├── poll-email/        # Outlook ポーリング Edge Function (5 分ごと
cron)
│       ├── auto-match/        # 自動マッチング Edge Function (毎朝 JST
9:00 cron・match-batch を内部呼び出し)
│       ├── match-batch/       # バッチ AI 採点 Edge Function (ルール事前フ
ィルタ + topN を 1 コール採点)
│       ├── match-score/       # 単発スコア Edge Function (duplicate 検出付
き・Cerebras→Groq→Gemini)
│       ├── archive-candidates/ # 7 日アーカイブ Edge Function (毎日 JST
0:00 cron・人材マップ全期間集計用)
│       ├── microsoft-oauth/   # Microsoft OAuth 認証 Edge Function
│       └── enrich-candidate/  # Box 連携・再解析 Edge Function (毎日 JST
3:00 cron)
│       └── skill-master-cleanup/ # skill_master クリーンアップ Edge Function
(毎日 cron)
├── public/
│   └── japan.topojson         # 47 都道府県の TopoJSON (人材マップ用・約 416KB)
├── scripts/
│   ├── verify_email_extraction.mjs # メール解析の品質検証用 Node スクリプト
│   ├── check-and-deploy-edge.sh   # deno check で TS2304 検知 → デプロイ
│   └── skill_master_review.py      # source='ai' スキルの月次レビュー
├── docs/
│   ├── Sales_Manual.md           # 営業担当者向け操作マニュアル
│   ├── HandsOn_Setup.md         # 環境構築ガイド (後任エンジニア向け)
│   ├── ai_fallback_flow.md      # AI フォールバックフロー詳細
│   ├── matching_candidate_selection.md # マッチング選定ロジック
│   ├── Heatmap.md              # 人材マップ (ヒートマップ) 機能仕様
│   ├── DataEnv_Demo_Prod.md     # データ環境 (prod/demo) の使い分け
│   ├── Outlook_AutoForward_Setup.md # Outlook 自動転送ルール設定
│   ├── AWS_Account_Setup_Guide.md # AWS アカウント作成 (参考資料)
│   ├── AI_Freetier_Challenges.md # 無料枠と現状の限界 (歴史的記述含む)
│   └── test-reports/           # テストレポート

```

問い合わせ・引き継ぎ

- GitHub: [kzmiyamura/akinavi-hr-ai-aws](#)
- Supabase プロジェクト ID: [argizomylbolpqxgmvim](#)